志愿填报作业 · 知识点 & 错题本（详解版）

> 这份文档是**根据我自己写作业时踩过的坑**整理的，不是标准答案。
> **怎么读**：
> 1. 先读「一、容器地图」——建立整体感觉
> 2. 再读「二、我踩过的 12 个坑」——这些是我真正犯过的错
> 3. 「三、知识点详解」是**原理层**的补充，第一遍不用全懂，写代码卡住时回来查
> 4. 查名词定义去「七、核心术语定义表」；忘了怎么写去「八、常用用法备注」
> 5. 最后用「五、交作业自检清单」过一遍

目录

- 一、五种数据容器：什么时候用哪个
- 二、我踩过的 12 个坑
- 三、知识点详解（进阶篇）
  - 3.1 可变 vs 不可变 3.2 引用与拷贝 3.3 哈希 3.4 容器选择流程 3.5 常用操作对照
  - 3.6 真值判断 3.7 比较与链式比较 3.8 `in` 的四种含义 3.9 切片完全指南 3.10 `range` 完全指南
  - 3.11 for / while / break / continue 3.12 边遍历边修改 3.13 推导式
  - 3.14 函数：参数 / 返回 / 作用域 3.15 函数是一等公民 3.16 默认参数陷阱
  - 3.17 常见报错速查 3.18 调试五步法 3.19 命名与注释规范 3.20 `__main__`
- 四、语法速查卡
- 五、交作业自检清单
- 六、三条通用心法
- 七、核心术语定义表
- 八、常用用法备注
- 附录：12 个任务 × 涉及知识点对照表

一、五种数据容器：什么时候用哪个

这是本次作业真正的考点。判断口诀：**要顺序用 list / tuple，要唯一用 set，要"按名字查东西"用 dict。**

容器	写法	三个关键词	本次作业用在哪
`list` 列表	`[1, 2, 3]`	有顺序、**能改**、可重复	`wish_list` 志愿表、`task_list` 任务清单、选科
`tuple` 元组	`(1, 2, 3)`	有顺序、**不能改**、可重复	`universities`（分数线是"事实"，不该被随手改）
`str` 字符串	`"张三"`	文字、**不可变**	`student["姓名"]`
`set` 集合	`{1, 2, 3}`	**自动去重**、**无序**、元素必须可哈希	兴趣标签去重、志愿表查重、推荐专业去重
`dict` 字典	`{"键": "值"}`	**按 key 找 value**	`student`、`major_map`、专业计数

两个容易忘的细节：

1. **list 和 tuple 唯一的根本区别**：能改 / 不能改。

试试 `universities[0][2] = 700` → 直接报错，这就是题目让你"体会"的地方。

2. **set 里不能放 list。** `set([[1,2],[3,4]])` 会报 `TypeError: unhashable type: 'list'`。

原因见 3.3 —— 集合靠"哈希值"判断元素是否重复，而**可变容器没法安全地哈希。**

思维导图：五种容器怎么选

【思维导图】以下为 Mermaid 源码，图形请在 Markdown 版查看

```
mindmap
 root((选哪个容器))
 一个东西
 str 文字
 int float 数字
 bool
 一堆东西
 要按名字查
 dict
 学生信息
 兴趣到专业映射
 专业计数
 要不能重复
 set
 兴趣标签去重
 志愿表查重
 推荐专业去重
 要能改
 list
 志愿表
 任务清单
 选科
 不能改 是事实
 tuple
 大学分数线
 函数多返回值
```

写代码前的「四问」流程

【思维导图】以下为 Mermaid 源码，图形请在 Markdown 版查看

```
flowchart TD
 A[拿到一份数据] --> B{是一个还是一堆}
 B -- 一个 --> C[str / int / float]
 B -- 一堆 --> D{要按名字查吗}
 D -- 要 --> E[dict]
 D -- 不要 --> F{要保证不重复吗}
 F -- 要 --> G[set]
 F -- 不要 --> H{需要改吗}
 H -- 需要 --> I[list]
 H -- 不需要 --> J[tuple]
```

12 个坑的因果图

【思维导图】以下为 Mermaid 源码，图形请在 Markdown 版查看

```
flowchart LR
 P1[坑1 链式比较] --> R1[结果恒为假]
 P2[坑2 pop 加下标自增] --> R2[只删一半]
 P3[坑3 没有 return] --> R3[结果变成 None]
 P5[坑5 判断写在循环里] --> R4[重复打印 N 次]
 P6[坑6 自己和自己比] --> R1
 P7[坑7 print 在循环外] --> R5[只打印最后一个]
 P8[坑8 函数不用参数] --> R6[换数据就失效]
 P9[坑9 用切片代替 for] --> R7[考点不给分]
```

二、我踩过的 12 个坑

坑 1：链式比较 —— `interest in major_map == True`

现象：推荐专业那一栏永远是空的，程序不报错，就是没结果。

```
if interest in major_map == True: # [×] 永远为 False
 recommended.append(interest)
```

为什么错：Python 允许**链式比较**，a < b < c 会被理解成 (a < b) and (b < c)。

in 也是**比较运算符**，所以上面那行实际是：

```
(interest in major_map) and (major_map == True)
# ↑ 字典永远不等于 True
```

结果永远是 False。实测：'计算机' in major_map == True → False，而 '计算机' in major_map → True。

正确写法：

```
if interest in major_map: # 直接写条件
 ...
```

一句话记住：**永远不要写** == True。if 条件：本身就是"如果条件为真"。

坑 2：pop + 下标自增 → 每删一个就跳过一个

现象：想把 task_list 清空，结果只删掉一半，剩下 ['查询大学', '提交志愿']。

```
i = 0
while i < len(task_list):
 task_list.pop(i) # [×]
 i += 1
```

为什么错：pop(i) 删掉一个元素后，**后面的元素会集体往前挪一格**，但 i 又往前走了，于是正好跳过下一个。手推一遍：

步骤	列表	说明
开始	[整理成绩, 查询大学, 筛选专业, 提交志愿]	i=0
第 1 轮后	[查询大学, 筛选专业, 提交志愿]	pop(0) 删掉"整理成绩"，i 变 1
第 2 轮后	[查询大学, 提交志愿]	pop(1) 删掉"筛选专业"，i 变 2
结束	[查询大学, 提交志愿]	len 只剩 2，i=2 不满足条件

正确写法（题目提示给了）：**永远删第 0 个**

```
while len(task_list) > 0: # 或者写成 while task_list:
 task_list.pop(0)
```

一句话记住：**边遍历边删除时，下标会"骗"你。**

坑 3：print ≠ return（最致命的一个）

现象：result = progress_function(0.05, final_analysis) 之后 print(result) 打出的是 None，精心写的报告一个字都没出现。

```
def progress_function(sleep_time, print_function):
 for x in range(100):
 ...
 sleep(0.1)
 if x == 99:
 print_function() # [×] 执行了，但返回值直接丢了
 # 函数最后没有 return → 返回 None
```

为什么错：**没有 return 的函数，返回值就是 None。**

- `print(x)` = 在屏幕上**显示** x，函数返回值还是 None
- `return x` = 把 x **交给调用者**，可以被 = 接住

`final_analysis()` 确实被调用了、也算出了报告字符串，但**没人接住 → 被扔掉了。**

正确写法：

```
return print_function() # 接住回调的返回值，再交出去
```

一句话记住：**想让外面接到东西，就必须** `return`；`print` **只是给人看的。**

坑 4：append 和 extend 分不清 / 存错了东西

现象：推荐专业里存的是"计算机"这种**兴趣标签**，而不是"计算机科学与技术"这种**专业。**

```
recommended_majors.append(interest) # [×] 存了标签
recommended_majors.extend(major_map[interest]) # [√] 存了专业
```

区别：

```
a = []
a.append(["计算机科学与技术", "软件工程"]) # → [['计算机科学与技术', '软件工程']] 整个列表塞进去
a.extend(["计算机科学与技术", "软件工程"]) # → ['计算机科学与技术', '软件工程'] 拆开逐个塞
```

一句话记住：append 塞**一个**东西，extend 把里面的东西**逐个**塞进去。

坑 5：把"整张表的判断"写在 for 循环里面

现象：程序连打 5 遍"志愿表中没有重复的大学名称"。

```
for i in range(len(wish_list)):
 if 有重复: # [×] 每所大学都判一次、打一次
 print("志愿表中存在重复大学")
```

为什么错：题目要判断的是"**整张志愿表**有没有重复"，一次就够，所以要**跳出循环**再判。

正确写法：

```
names = [item[0] for item in wish_list]
if len(set(names)) == len(names): # 循环之外，只判一次
 print("志愿表中没有重复大学")
else:
 print("志愿表中存在重复大学")
```

一句话记住：**循环负责"收集数据"，判断和输出放在循环外面。**

坑 6：拿"自己和自己比"，条件恒为真

现象：不管资料里有没有重复，永远输出"没有重复"。

```
if list(set(wish_list[i][0])) == list(set(wish_list[i][0])): # [×] 左右完全一样
```

为什么错：等号两边是**同一个东西**，永远相等。我本来想比较"原始"和"去重后"，但两边都写了去重版。

正确的查重思路 —— 比"去重前后的个数"：

```
len(set(names)) == len(names)
```

- 5 个名字 → 放进 set → 还剩 5 个 → 一个都没被去掉 → **没重复**
- 5 个名字 → 放进 set → 变成 4 个 → 有 2 个一样 → **有重复**

一句话记住：**查重 =** `len(set(x))` **和** `len(x)` **比大小。**

坑 7：print 写在 for 外面 → 只打印最后一个

现象：专业计数只输出了一行 专业 电子信息工程：出现 2 次，其他专业都消失了。

```
for i in range(len(wish_list)):
 for major in wish_list[i][1]:
 major_count[major] = major_count.get(major, 0) + 1
print(f"专业 {major}：出现 {major_count[major]} 次") # [×] 在循环外！
```

为什么错：统计部分其实是**对的**，但打印跑出了循环，所以只在最后打印了一次，用的还是循环结束时残留的那个 major。

正确写法 —— 用 for 遍历字典：

```
for major in major_count:
 print(major, "出现了", major_count[major], "次")
```

一句话记住：**"每一个"要打印 → print 必须在 for 里面。**

坑 8：函数写了参数，却根本没用

现象：analyze_student(student, universities) 看起来没问题，但函数体里从来没用过这两个参数。

```
def analyze_student(student, universities):
 return challenge_list, safe_list, not_recommended_list # [×] 用的是外面的全局列表
```

为什么错：**参数是函数的"输入"**。函数应该"给我什么，我算什么"，而不是偷偷去读外面的全局变量。否则这个函数换个学生、换份学校表就完全没用了。

正确写法 —— 用参数，在函数内部新建列表：

```
def analyze_student(student, universities):
 challenge, safe, not_recommended = [], [], []
 for name, _city, line in universities:
 if student["高考分数"] >= line:
 safe.append(name)
 elif student["高考分数"] >= line - 10:
 challenge.append(name)
 else:
 not_recommended.append(name)
 return challenge, safe, not_recommended
```

另一个隐藏问题：直接返回全局列表时，外面任何地方改了那个列表，"函数返回的结果"也会跟着变（它们是**同一个对象**，原理见 3.2）。

一句话记住：**函数只认参数，不认全局。**

坑 9：任务 4 用切片代替了 for 循环

现象：题目要求"**使用 for 循环**倒序复制，并且子列表也要倒序"，我写的是 new_list = my_list[::-1]。

为什么错：

- 题目写"**必须使用 for 循环**"——这是考点，用了切片**直接不给分**
- my_list[::-1] 只能把**外层**倒过来，里面的 ["计算机","机器人","自动化"] 还是原顺序，**没满足"子列表也倒序"**

正确思路：外层用 range 倒着走，碰到子列表再用切片倒一次。

```
new_list = []
for i in range(len(my_list) - 1, -1, -1): # 外层：整体倒序
 item = my_list[i]
 if isinstance(item, list): # 判断是不是列表
 new_list.append(item[::-1]) # 子列表也倒序
 else:
 new_list.append(item)
```

一句话记住：**作业里写"必须用 X"的地方，就用 X，别用更聪明的写法。**

坑 10：range 的"左闭右开"没搞清

现象：倒着遍历时少走了一格 / range 的终点写错。

规则：range(起点，终点，步长)，**包含起点，不包含终点。**

```
range(5) # 0 1 2 3 4
range(2, 5) # 2 3 4 终点 5 取不到
range(0, 10, 2) # 0 2 4 6 8
range(4, -1, -1) # 4 3 2 1 0 倒着走
```

所以倒着遍历**全部下标**要写 range(len(x) - 1, -1, -1)：

- 起点 len-1 = 最后一个下标
- 终点写 -1（因为右开，走到 -1 才会包含 0）
- 步长 -1 = 每次减 1

一句话记住：**终点取不到，所以倒着走要写到 -1。**

坑 11：注释掉的"半成品代码"没清掉

现象：文件里留着这样一段被 # 包起来的死代码：

```
# for x in my_list:
# if isinbstance(x, list)==True ← 拼错了，而且没有冒号
# x=x[::-1]
# if 学生分数 >= 大学分数线: ← 变量根本不存在
```

问题：isinbstance 拼错了（正确 isinstance），学生分数
这个变量从来没定义过，缩进也乱了。这种"写了一半就放弃"的代码会让人以为任务没完成。

正确做法：要么真正写好，要么彻底删掉。**交作业前搜一遍拼写错误。**

坑 12：变量名拼错 + 变量写完全没用

现象：

- recomended_majors → 应为 recommended_majors
- appilication → 应为 application
- major_pool、选科、目标城市 定义了却一次都没用上

为什么要注意：文档的「优秀作业标准」明确写了"**变量名有意义**"。

一句话记住：**变量名是给自己三个月后看的，别偷懒。**

三、知识点详解（进阶篇）

> 这一章把前面 12 个坑背后**真正的原理**补全。第一次看不用全懂，遇到问题回来查就行。
> 建议顺序：**3.1 → 3.2 → 3.3**（容器三大原理），其余按需查。

3.1 可变 vs 不可变 —— Python 最底层的一条分界线

不可变类型（immutable）：int、float、str、tuple、bool、frozenset、None

可变类型（mutable）：list、dict、set、bytearray

"不可变"到底是什么意思？

不是"你不能改"，而是"**它不允许被原地修改**"——你只能造一个新的对象，把变量名重新指过去。

```
s = "abc"
s[0] = "A" # [×] TypeError: 'str' object does not support item assignment

s = s + "d" # [√] 这句话其实是"新建了 abcd 这个新字符串"，再让 s 指向它
```

```
t = (1, 2, 3)
t[0] = 9 # [×] TypeError: 'tuple' object does not support item assignment

lst = [1, 2, 3]
lst[0] = 9 # [√] 列表可以原地改
lst.append(4) # [√] 原地追加
```

怎么自己验证一件事会不会"原地改"？

用 id() 看内存地址，变没变一目了然：

```
lst = [1, 2, 3]
print(id(lst)) # 比如 140234...
lst.append(4)
print(id(lst)) # 地址**没变** → 原地修改（mutable）

s = "abc"
print(id(s))
s = s + "d"
print(id(s)) # 地址**变了** → 新建了对象（immutable）
```

Python 为什么要这样设计？

因为**不可变 = 安全 = 可以当"标签"用。**回想坑 1 的 set：集合和字典的 key 都要求"**可哈希（hashable）**"，而可哈希的前提就是**不可变。**如果 list 能当 set 的元素，你往列表里加个东西，集合里存的"标签"就失效了。所以 Python 干脆禁止。

对本作业的意义：universities 用 tuple 装，就是想表达"**这些录取分数线是既定事实，代码里不应该修改它**"。这是一种"用类型表达意图"的写法。

3.2 变量是"名字"，不是"盒子" —— 引用、is、拷贝

这是初学者最容易误解的一点。**在 Python 里，变量不装值，变量只是贴在对象上的一个名字。**

```
a = [1, 2, 3]
b = a # ← 注意！这不是"复制一份"，而是"给同一个列表又起了个名字"
b.append(4)
print(a) # [1, 2, 3, 4] ← a 也跟着变了！
print(a is b) # True ← 它们指的是同一个对象
```

内存里的样子：

```
a ─┐
   ├──→ [1, 2, 3, 4] ← 只有"一个"列表对象
b ─┘
```

is 和 == 的区别

写法	比较的是	例子
a == b	**值**是否相等	[1,2] == [1,2] → True
a is b	是不是**同一个对象**（地址相同）	[1,2] is [1,2] → False

```
x = [1, 2]
y = [1, 2]
print(x == y) # True 值一样
print(x is y) # False 两个不同的列表对象
print(id(x), id(y)) # 两个不同的地址
```

> ※ 一句话：is **问"是不是本人"，** == **问"长得一样吗"。**
> 只有判断 None 才习惯用 is：if x is None:

怎么真正"复制"一份?

```
a = [1, 2, 3]

b = a # [×] 只是别名，不复制
b = a.copy() # [√] 浅拷贝
b = list(a) # [√] 另一种浅拷贝
b = a[:] # [√] 切片也能复制整个列表
```

[!] 浅拷贝的坑：里层不复制

```
a = [1, 2, [3, 4]]
b = a.copy() # 浅拷贝：只复制了最外面一层
b[0] = 99
print(a) # [1, 2, [3, 4]] ← 外层安全，a 没变
b[2].append(5)
print(a) # [1, 2, [3, 4, 5]] ← 里层共享！a 也变了
```

真要把里层也复制干净，得用**深拷贝**：

```
import copy
b = copy.deepcopy(a) # 从里到外全部复制一份
```

这个知识点直接解释了坑 8

```
def analyze_student(student, universities):
 return challenge_list, safe_list, not_recommended_list # [×] 返回的是全局对象本身
```

因为返回的是**同一个列表**，所以外面任何人 safe_list.append(...)，函数"返回的结果"也会跟着变——它们本来就是一个东西。**在函数内部新建列表再返回**，才是干净的写法。

顺便： is **的一个记忆陷阱**

Python 对小整数（-5~256）和短字符串做了缓存，所以偶尔会出现：

```
a = 256; b = 256; print(a is b) # True（缓存）
a = 1000; b = 1000; print(a is b) # 可能是 False!
```

这就是为什么**永远不要用** is **去比较数字或字符串**，比数字就用 ==。

3.3 哈希（hashable）—— 为什么 set 里不能放 list

"可哈希"= 能算出一个固定的"身份证号"，集合和字典就靠这个号来快速判断"你有没有出现过"。

```
hash("abc") # [√] 可以，返回一个整数
hash((1, 2)) # [√] 元组可以（前提是里面元素也都可哈希）
hash([1, 2]) # [×] TypeError: unhashable type: 'list'
hash({"a": 1}) # [×] 同样不行
```

所以：

```
set([[1, 2], [3, 4]]) # [×] 报错
set([(1, 2), (3, 4)]) # [√] 元组可以
```

规律：可变的东西一般不可哈希，不可变的一般可哈希。

这条和 3.1 是同一件事的两种说法。

它还决定了 dict 的 key 能用什么：

```
d = {}
d["张三"] = 90 # [√] 字符串作 key
d[(1, 2)] = "坐标" # [√] 元组作 key
d[[1, 2]] = "列表" # [×] 列表不能作 key
```

3.4 容器选择：一套可以照着走的决策流程

写代码前，按顺序问自己这四个问题：

1. **这是"一个东西"还是"一堆东西"？**
   - 一个东西 → 用 `str` / `int` 等基本类型
   - 一堆东西 → 继续往下问
2. **需要"按名字查"吗？**（比如 `student["姓名"]`、专业出现次数）
   - 要 → 用 `dict`
3. **需要"不能重复"吗？**（去重、查重）
   - 要 → 用 `set`
4. **剩下的：需要能修改吗？**
   - 能改 → `list`
   - 不能改（是"事实 / 常量"）→ `tuple`

放到本次作业里对照：

- `student`：要按名字查"姓名""高考分数" → **dict**
- `universities`：一堆"事实"，不该被改 → **tuple**（里面每条也是 tuple）
- `wish_list`：有顺序、可能要增删 → **list**（里面嵌套 list）
- 兴趣标签去重、推荐专业去重、志愿表查重 → **set**
- `major_map`、专业出现次数 → **dict**
- `student["姓名"]` → **str**

3.5 五种容器常用操作对照表

list 列表

操作	写法	说明
加一个	`lst.append(x)`	加到最后
加一堆	`lst.extend([a, b])`	逐个加进去
指定位置插入	`lst.insert(0, x)`	插到最前面
按下标删	`lst.pop(0)`	删掉并**返回**它
删最后一个	`lst.pop()`	最常用
按值删	`lst.remove("提交志愿")`	删第一个匹配的值

操作	写法	说明
全清空	`lst.clear()`	本作业禁止用
查位置	`lst.index(x)`	返回下标，找不到报错
数个数	`lst.count(x)`	出现几次
排序	`lst.sort()`	**原地**排，返回 None
倒过来	`lst.reverse()`	**原地**倒，返回 None
复制	`lst.copy()`	浅拷贝

> [!] `sort()` / `reverse()` 是**原地修改**，返回值是 `None`。
> 所以 `lst = lst.sort()` 是错的（lst 会变成 None）！要新列表就用内置函数 `sorted(lst)` / `list(reversed(lst))`。

tuple 元组：只能"查"，不能"改"

```
universities[0] # ("清华大学", "北京", 700)
universities[0][2] # 700
len(universities) # 6
```

想改 → 只能造个新的元组。

str 字符串

```
s.strip() # 去首尾空白
s.replace(" ", "") # 替换
s.split(",") # 按分隔符切成列表
"、".join(["A", "B"]) # 把列表拼成字符串
s.startswith("张") # 是不是以...开头
s.upper() / s.lower() # 大小写
s.find("三") # 找位置，找不到返回 -1
"三" in s # 子串判断
len(s) # 长度
```

set 集合

```
st.add(x) # 加元素
st.remove(x) # 删（不存在会报错）
st.discard(x) # 删（不存在也不报错）
st & other # 交集
st | other # 并集
st - other # 差集
len(st) # 元素个数
x in st # 判断有没有（很快）
```

dict 字典

```
d["姓名"] # 取（键不存在会 KeyError）
d.get("年龄", "未知") # 取（取不到返回默认值，不报错）
"姓名" in d # 判断"键"在不在
d.keys() # 所有键
d.values() # 所有值
d.items() # 所有键值对 → for k, v in d.items():
d[key] = value # 新增或修改
del d[key] # 删除
```

> ※ 记住：`in` **对字典判断的是"键"，不是"值"。**
> `"张三" in student` 判断的是有没有叫"张三"这个**键**，不是有没有这个值。

3.6 真值判断：什么算"假"

Python 里 `if` / `while` 后面不一定非得是 `True`/`False`，任何东西都能判真假：

会当成"假"的：

```
False, None, 0, 0.0, "", [], (), {}, set()
```

其余几乎都是"真"（包括 -1、"False"、[0]）。

所以这几组是等价的：

```
if len(task_list) > 0: ≡ if task_list:
if len(s) == 0: ≡ if not s:
while len(lst) > 0: ≡ while lst:
```

这也解释了一个隐患：如果某个值可能是 0，就别写 if x: 来判断"有没有值"，因为 0 会被当成"没有"。

```
score = 0
if score: # [×] 0 会被当成 False，进不去
 print("有分数")
```

3.7 比较运算符与链式比较

基本比较：== != > < >= <=

[!] **易错**：赋值是 =，比较是 ==。写错不会报错（把比较结果赋给变量了），但逻辑全错。

链式比较是 Python 的特色，也是坑 1 的根源：

```
0 <= score <= 100 # [√] 等价于 (0 <= score) and (score <= 100)
```

规则：a op1 b op2 c ≡ (a op1 b) and (b op2 c)，**中间那个只算一次。**

参与链式比较的运算符包括：< > <= >= == != **以及** in / not in / is / is not。

所以坑 1 那行才会变成"和 True 比较"：

```
if interest in major_map == True:
# ↓ 等价于
if (interest in major_map) and (major_map == True): # 后半永远为 False
```

防坑写法：**永远不要写** == True / == False，直接用值本身。

3.8 in 的四种含义（很容易混）

in 在不同容器上判断的东西**不一样**：

写法	判断的是	例子
"三" in "张三"	**子串**	→ True
"计算机" in ["计算机", "AI"]	**元素**	→ True
"姓名" in student (dict)	**键**（不是值！）	→ True
"计算机" in major_map (dict)	**键**	→ True
"计算机" in {"计算机", "AI"} (set)	**元素**	→ True

坑 1 里我用 interest in major_map
判断"这个兴趣标签有没有对应的专业"，用的正是**字典判键**，方向是对的——只是被 == True 毁了。

3.9 切片（slicing）完全指南

写法：容器[起点 : 终点 : 步长]，**左闭右开**，三个都可以省略。

```
lst = ["a", "b", "c", "d", "e"]

lst[1:3] # ['b', 'c'] 从 1 到 3（不含 3）
lst[:2] # ['a', 'b'] 省略起点 = 从头开始
lst[2:] # ['c', 'd', 'e'] 省略终点 = 到末尾
lst[:] # 整个列表的"浅拷贝"
lst[-2:] # ['d', 'e'] 负数下标：从倒数第 2 个到末尾
lst[:-2] # ['a', 'b', 'c'] 去掉最后两个

lst[::-1] # ['e','d','c','b','a'] 倒序 ← 本作业常用
lst[::2] # ['a', 'c', 'e'] 隔一个取一个
lst[4:1:-1] # ['e', 'd', 'c'] 倒着取
```

几个要点：

1. **字符串也能切片**："张三丰"[1:] → "三丰"
2. **切片越界不报错**，会自动截断：lst[0:100] → 整个列表
3. **但普通下标越界会报错**：lst[100] → IndexError
4. [::-1] 是**倒序**的经典写法，但**本作业任务 4 / 挑战 2 明确要求用 for + range**，别用切片

3.10 range 完全指南

```
range(stop) # 0, 1, ..., stop-1
range(start, stop) # start, ..., stop-1
range(start, stop, step) # 按步长走
```

```
list(range(5)) # [0, 1, 2, 3, 4]
list(range(1, 5)) # [1, 2, 3, 4] 左闭右开!
list(range(0, 10, 3)) # [0, 3, 6, 9]
list(range(5, 0, -1)) # [5, 4, 3, 2, 1]
list(range(4, -1, -1)) # [4, 3, 2, 1, 0] 倒着走完 0
```

倒序遍历下标的黄金公式：

```
for i in range(len(lst) - 1, -1, -1):
 ...
```

拆开看：len(lst) - 1 是最后一个下标；-1 是终点（右开，所以 0 会被包含）；最后的 -1 是步长。

[!] range 本身不是一个列表，是"惰性"的，直接 print(range(5)) 会输出 range(0, 5)，想看内容要 list(range(5))。

3.11 for / while / break / continue

什么时候用哪个？

- **已知要跑多少次 / 要遍历一个容器的每一个元素** → 用 for
- **不知道要跑多少次，只知道"满足某条件就一直跑"** → 用 while

本次作业里的例子：

任务	用哪个	为什么
任务 3 清空列表	while	不知道要循环几次，只知道"还有元素就继续"
任务 4 倒序复制	for + range	明确要遍历每个下标一次
任务 5 分档	for	遍历 universities 每个元素
挑战 3 计数	for	遍历每所大学、每个专业

break **和** continue：

```
for x in [1, 2, 3, 4, 5]:
 if x == 3:
 continue # 跳过这一次，继续下一轮（3 不打印，4、5 照打）
 if x == 5:
 break # 直接跳出整个循环
 print(x) # 输出 1 2 4
```

`for ... else`**（少见但要认得）**：`else` 在"循环正常跑完、没被 break 打断"时执行。

```
for x in lst:
 if x == 目标:
 print("找到了")
 break
else:
 print("没找到") # 只有循环没被 break 才执行
```

还有个坑：`for` 的循环变量在循环结束后**还存在**（坑 3 里的 `x` 就是这么来的）。所以别在循环外依赖它——如果容器是空的，它根本没被定义过。

3.12 边遍历边修改 —— 一个必须警惕的操作

坑 2 的本质。看这个"看起来没问题"的写法：

```
lst = ["a", "b", "c", "d"]
for x in lst:
 if x in ("a", "b"):
 lst.remove(x)
print(lst) # ['b', 'd'] ← 不是期待的 ['c', 'd']!
```

为什么：`for` 内部是按**下标**往前走的；你删掉一个元素，后面的元素集体前移，序号就错位了。

三种正确做法：

```
# ① 遍历"副本"，改的却是原列表
for x in lst[:]:
 lst.remove(x)

# ② 倒着遍历（倒着删不会影响还没处理的下标）
for i in range(len(lst) - 1, -1, -1):
 if 条件:
 lst.pop(i)

# ③ 直接重新构造（最推荐，最不容易错）
lst = [x for x in lst if not 条件]
```

一句话：要删东西，就倒着遍历，或者遍历副本，或者重建列表。

3.13 列表推导式（认得就好，本作业别用）

```
# 传统写法
squares = []
for x in range(5):
 squares.append(x * x)

# 推导式写法（等价）
squares = [x * x for x in range(5)]
```

带条件的：

```
names = [item[0] for item in wish_list] # 挑战 1 里取所有大学名
big = [x for x in nums if x > 10] # 只保留大于 10 的
new = [x[::-1] if isinstance(x, list) else x for x in my_list] # 带 if/else 的
```

但是：任务 4 和挑战 2 **明确要求用** for + range，就是为了考你基本功，所以那里**不许**用推导式或切片偷懒。

3.14 函数：参数、返回值、作用域

函数的最小模型：你给我输入（参数），我还你输出（返回值）。

```
def 函数名(参数1, 参数2):
 """文档字符串：一句话说明这个函数干什么。"""
 处理...
 return 结果
```

① 参数：函数的"入口"

```
def analyze_student(student, universities): # 两个入口
 ...
```

调用时按顺序传进去：analyze_student(student, universities)。

也可以按名字传（更清楚）：analyze_student(universities=universities, student=student)。

② 返回值：函数的"出口"

```
def f():
 print("我执行了") # 只是显示，返回值是 None

def g():
 print("我执行了")
 return 42 # 返回 42

print(f()) # 打印"我执行了"，然后打印 None
print(g()) # 打印"我执行了"，然后打印 42
```

没有 return 的函数，返回值就是 None（坑 3 的根源）。

return 还有两个作用：**立刻结束函数**（后面的代码不再执行）、**把值交给调用者。**

③ 多返回值 = 打包成元组

```
def analyze_student(...):
 return challenge_list, safe_list, not_recommended_list # 本质是返回一个元组

结果 = analyze_student(...)
print(结果) # (['上海交通大学'], [...], ['清华大学'])
print(结果[0]) # 也可以按下标取

# 更常用的写法：解包
challenge, safe, not_recommended = analyze_student(...)
```

④ 作用域（LEGB）：变量按 **局部 → 外层 → 全局 → 内置** 的顺序查找。

```
score = 686 # 全局变量

def show():
 print(score) # [√] 读全局：允许

def change():
 score = 700 # [!] 这不会改全局！只是新建了一个"局部变量"
 print(score) # 700（局部的）

change()
print(score) # 686（全局没变）
```

真想改全局要在函数里声明 global score——**但强烈不推荐**，那会让函数"偷偷改外面的世界"，很难调试。**正确做法：需要什么当参数传进来，算完用**

return 送出去。

这也正是坑 8 的教训。

3.15 函数是"一等公民" —— 可以当参数传（回调函数）

这是坑 3 / 任务 9 的关键。

```
def say_hello():
 print("你好")

def run_twice(func): # 参数是个函数
 func()
 func()

run_twice(say_hello) # [√] 传"函数本身"（没有括号）
```

秘诀就一句话：函数名后面加不加括号，含义完全不同。

写法	含义
say_hello	**函数对象本身**（一个"待执行的动作"）
say_hello()	**立刻执行它**，得到它的返回值

所以任务 9：

```
result = progress_function(0.05, final_analysis) # [√] 传进去，等进度条跑完再执行
result = progress_function(0.05, final_analysis()) # [×] 先执行完，再把结果传进去
```

progress_function 的写法就是：

```
def progress_function(sleep_time, print_function):
 ...画进度条...
 return print_function() # ← 在合适的时机"调用"它，并且把返回值交出去
```

> ● 顺便：my_list 里同时出现了 progress_function（函数本身）和 progress_function(0.05, print)（立刻执行）。这是题目故意留的对比。

3.16 默认参数的陷阱（进阶，知道就好）

```
def add_item(x, lst=[]): # [×] 危险！默认值只在定义时创建一次
 lst.append(x)
 return lst

print(add_item(1)) # [1]
print(add_item(2)) # [1, 2] ← 咦？它不是应该重新开始吗？
```

原因：默认参数 [] 是在**函数定义时**创建的**同一个列表**，被多次调用共享。

正确写法：用 None 当默认值，在函数里面再创建。

```
def add_item(x, lst=None):
 if lst is None:
 lst = []
 lst.append(x)
 return lst
```

> 本次作业不会遇到，但以后写函数一定会踩，先记住。

3.17 常见报错速查表（看报错不再发懵）

Python 的报错信息**从下往上看**：最后一行是"错误类型 + 原因"，倒数第二行是"出错的文件和行号"。

报错	中文意思	常见原因	怎么修
SyntaxError	语法错误	少了冒号 :、括号 / 引号不配对	看它标的位置**前一行**
IndentationError	缩进错误	该缩进没缩进，或者空格/Tab 混用	统一用 4 个空格
NameError	名字不存在	变量名拼错、变量还没定义就用了	检查拼写；确认赋值在调用之前
TypeError	类型不对	"年龄" + 18、len(123)、unhashable type: 'list'	先 str() / int() 转换；set/dict 别放 list
ValueError	值不对	int("abc")、"1,2".split(",") 用法错	检查内容格式
IndexError	下标越界	lst[5] 但列表只有 3 个元素	用 len(lst) 检查范围
KeyError	字典没有这个键	d["不存在的键"]	先用 in 判断，或用 d.get(键，默认值)
AttributeError	对象没有这个属性/方法	方法名拼错（lst.apppend()），或类型不对	检查拼写、检查对象类型
ZeroDivisionError	除以 0	10 / 0	加判断
UnboundLocalError	局部变量还没赋值就用	函数里既赋值又读同名全局变量	见 3.14 作用域

读报错的固定动作：看最后一行知道"错在哪类"，看倒数第二行知道"错在哪一行"，然后去那一行。

3.18 调试五步法

1. **读报错**：错误类型 + 行号（从下往上看）
2. **去那一行**：看这一行用了哪些变量
3. **打印它们**：print(变量, type(变量))，看值和类型对不对
4. **缩小范围**：把没问题的部分先注释掉，二分定位
5. **手推一遍**：拿纸画表格，把每一步的变量值写下来（坑 2 就是这么找到的）

不报错但结果不对时，第 5 步最关键——因为这类 bug（坑 1、坑 6）**只能靠对照题目要求的"目标效果"发现。**

print **调试模板**：

```
print("DEBUG 分数 =", student["高考分数"], "类型 =", type(student["高考分数"]))
print("DEBUG 稳妥列表 =", safe_list)
print("DEBUG major_map 里有计算机吗:", "计算机" in major_map)
```

> VS Code 里也可以点行号左边打**断点**，然后按 F5 单步执行，看变量面板——比 print 更直观，值得学。

3.19 命名与注释规范（对应"优秀作业标准"）

命名

- 变量 / 函数：小写_加下划线（snake_case），如 safe_list、analyze_student
- 常量（不会变的值）：全大写，如 MAX_SCORE = 750
- **要有意义**：safe_list 比 a、list1 好一万倍
- 不要用拼音、不要用中文变量名
- 别用 Python 内置名字当变量名：list、dict、sum、max、str、input、print
- 单字母只用在短循环：for i in range(...)

注释

- 函数用 **docstring**（三引号），说明：**做什么、参数是什么、返回什么**
- 行内注释解释 **为什么这么做**，而不是复述代码
  - [×] `i += 1 # i 加 1`
  - [√] `i += 1 # 指向下一个下标（注意：删除元素后要重新想清楚下标怎么走）`
- 作业要求"**所有函数和代码实现都需要有详细注释**"，别偷懒
- **别留一大片注释掉的废代码**（坑 11）

格式（PEP 8 摘要）

- 缩进统一 4 个空格
- 逗号后面加一个空格：`[1, 2, 3]`
- 运算符两边加空格：`a = b + c`
- 函数之间空 2 行

3.20 `if __name__ == "__main__":` 是干什么的

```
def final_analysis():
 ...

if __name__ == "__main__":
 result = progress_function(0.05, final_analysis)
 print(result)
```

- **直接运行** `python 文件.py` 时，Python 会给这个文件起名叫 `"__main__"` → `if` 成立 → 里面的代码执行
- **被别的文件** `import` 时，`__name__` 是模块名（比如 `"高考志愿冒险模拟器"`）→ `if` 不成立 → 里面的代码**不执行**

好处：别人想复用你写的 `analyze_student` 函数时，`import` 一下不会把整个程序连带跑一遍。

> 小实验：在文件最后加一行 `print("模块名是", __name__)`，直接运行会打印 `__main__`。

四、语法速查卡

```
# ---------- 字符串 ----------
s.strip() # 去首尾空白（记得用 = 接住！）
s.lstrip() / s.rstrip() # 只去左边 / 只去右边
s.replace(" ", "") # 去掉所有空格
s.split(",") # 按分隔符切成列表
"、".join(["A", "B"]) # → "A、B"（列表里必须都是字符串）
s.startswith("张") # 开头判断
len(s) # 长度

# ---------- 列表 ----------
lst.append(x) # 尾部加一个
lst.extend([a, b]) # 把里面的逐个加进去
lst.insert(0, x) # 插到指定位置
lst.pop() # 删最后一个并返回
lst.pop(0) # 删第一个
lst.remove(x) # 按值删第一个匹配的
lst.clear() # 全清空（本作业任务 3 禁止用）
lst.index(x) / lst.count(x)

# ---------- 切片 ----------
lst[::-1] # 整个倒过来
lst[a:b] # 左闭右开
lst[start:stop:step] # 完整写法
lst[:] # 浅拷贝一份

# ---------- range ----------
range(n) # 0 .. n-1
range(a, b) # a .. b-1
range(len(x) - 1, -1, -1) # 下标倒着走

# ---------- set ----------
len(set(x)) == len(x) # 判断有没有重复
st.add(v) # 加元素
list(set(x)) # 转回列表（顺序会变）
```

```
# ---------- dict ----------
d[key] # 取不到会 KeyError
key in d # 先判断在不在
d.get(key, 默认值) # 取不到给默认值
d.items() # 遍历"键, 值"

# ---------- 字典计数（背下来）----------
count = {}
for 元素 in 一堆元素:
 if 元素 in count:
 count[元素] += 1
 else:
 count[元素] = 1
# 简写：count[元素] = count.get(元素, 0) + 1

# ---------- 类型判断 ----------
isinstance(x, list) # x 是不是列表

# ---------- 元组解包 ----------
for name, city, line in universities: # 一行拆开三个
challenge, safe, not_rec = analyze_student(...) # 多返回值也是解包

# ---------- 其他常用内置函数 ----------
len(x) / sum(x) / max(x) / min(x)
sorted(lst) # 返回排好序的"新列表"（不改原列表）
list(reversed(lst)) # 倒序的新列表
enumerate(lst) # for i, v in enumerate(lst):
zip(a, b) # for x, y in zip(a, b):
str(686) / int("686") # 类型转换
```

五、交作业自检清单

- ■ 任务 4 用的是 **for + range**，不是 `[::-1]` 切片
- ■ 任务 2 用了 **set**；挑战 2 用了 **range 倒序**（"必须用"的地方别换写法）
- ■ 任务 9 的 `print(result)` 能打出**完整报告**，不是 `None`
- ■ 挑战 1 只打印**一次**，文案是「志愿表中存在重复大学 / 志愿表中没有重复大学」
- ■ 挑战 3 能打印**每一个**专业
- ■ 每一段代码都有**详细注释**（文档明确要求）
- ■ 加了**生成式 AI 辅助声明**（文档第一条要求，按真实情况写）
- ■ 没有拼写错误的变量名，没有一片注释掉的废代码
- ■ 没有把 `list` / `dict` / `sum` 这些内置名字当变量名
- ■ `python 文件名.py` 能一次跑通、不报错
- ■ 打包成 `.zip`；邮件标题 `第3次作业_姓名_省份` → `EL@zju.edu.cn`

六、三条通用心法

1. 报错不可怕，"不报错但结果是错的"才可怕。

坑 1（链式比较）、坑 6（自己和自己比）都不报错，却让结果全错。

改完代码**一定要看输出**，对着题目要求的"目标效果"逐项核对。

2. 分清 `print` / `return` / "调用"。

- `print(x)`：显示
- `return x`：交出去
- `f` 是"动作"，`f()` 才是"执行这个动作"

3. 先想"这是什么容器、我该拿它做什么"，再动手写。

写代码前先问：这个数据是会变的列表，还是不变的元组？要查重吗？要按名字找吗？

——想清楚这一步，这次作业 80% 的坑都不会踩。

七、核心术语定义表

> 看懂术语，读文档、搜资料、看报错都会快很多。**备注栏是"实战里要注意什么"。**

术语	定义	常用备注
变量	贴在对象上的一个"名字"，靠它访问对象	变量本身**没有类型**，类型属于它指向的对象；`a = b` 不是复制
对象	内存里真实存在的数据实体	`id()` 看身份，`type()` 看类型
引用	变量"指向"某个对象的关系	两个变量可指向同一对象，改一个另一个也变（见 3.2）
可变 / 不可变	能不能被"原地修改"	可变：`list dict set`；不可变：`str int float tuple bool None`
哈希（hashable）	能算出一个固定哈希值的性质	可哈希才能当 set 元素 / dict 的 key；`list dict set` 不可哈希
字面量（literal）	代码里**直接写出来**的值	`[1,2]` 列表、`(1,2)` 元组、`{"a":1}` 字典、`{1,2}` 集合、`"abc"` 字符串
表达式 / 语句	表达式**有值**，语句是**一个动作**	`1 + 1` 是表达式；`if` / `for` / `print()` 是语句
下标（索引）	用位置取元素	从 **0** 开始；`lst[-1]` 是最后一个；越界报 `IndexError`

术语	定义	常用备注
切片（slice）	用 [起:止:步] 取一段	**左闭右开**；越界不报错；[::-1] 倒序
可迭代对象	能被 for 逐个取出的东西	list / tuple / str / dict / set / range 都可以
迭代	用 for 逐个访问的过程	遍历 dict 得到的是**键**，不是值
解包（unpacking）	把容器元素一次分给多个变量	a, b = (1, 2)；**函数多返回值**也靠它
真值判断	一个值"算真还是算假"	假值只有这几种：False None 0 0.0 "" [] () {} set()
函数（function）	一段可重复使用的代码，有输入有输出	没有 return 就返回 None
参数 / 实参	定义时写的是**参数**，调用时传的是**实参**	参数是函数的"入口"，别用它偷偷读全局变量
返回值	return 交出去的结果	print ≠ return（坑 3）
作用域（scope）	变量能被访问到的范围	查找顺序 LEGB；函数里赋值默认是**局部**
回调函数（callback）	当作参数传进去、由别人在合适时机调用的函数	progress_function(0.05, final_analysis)
一等公民	函数能像普通值一样被赋值、传递、返回	所以函数能放进列表（my_list 里就有）
方法 / 函数	lst.append(x) 是**方法**（长在对象上）；len(lst) 是**函数**	方法写作 对象.方法名()
原地修改（in-place）	直接在原对象上改，不产生新对象	append / sort / reverse / clear 都是原地，**返回** None
浅拷贝 / 深拷贝	浅：只复制最外层；深：连里层一起复制	a.copy() 是浅拷贝；copy.deepcopy(a) 才是深
推导式（comprehension）	用一行"描述"生成列表/字典/集合	本作业任务 4、挑战 2 **禁止**用它代替 for
模块（module）	一个 .py 文件	from progress_function import progress_function
异常（exception）	程序运行时出的错	读报错：**最后一行**是错误类型，**倒数第二行**是位置
None	Python 表示"什么都没有"的值	判断用 is None，不要写 == None
语法糖	写起来更方便的等价写法	切片、推导式、多返回值、a, b = b, a 都是

八、常用用法备注（按类型速查）

> 这一节是"**平时写代码时最常忘的那些点**"。符号含义：
> [√] 推荐这么写 [!] 要注意 [×] 别这么写

8.1 字符串 str

定义：一串字符，**不可变**（不能原地修改）。

- [√] 去空格后**一定要接住**：s = s.strip()，只写 s.strip() 等于没做
- [√] 用 ""、"".join(lst) 拼多个字符串，比用 + 更清晰也更快
- [√] join 的列表元素**必须全是字符串**，数字要先 str(x)（如 str(686)）
- [!] .replace(a, b) 默认替换**全部**匹配，不是只换第一个
- [!] .split() 不传参数 = 按**任意空白**切；.split(",") 按逗号切
- [!] .strip() 只去**首尾**，不去中间的字符
- [×] 别在循环里 s = s + x 拼长字符串（每次都在造新对象，很慢）

8.2 列表 list

定义：**有顺序、可修改、可重复**的序列。

- [√] 加**一个**用 `append(x)`，加**一批**用 `extend([...])`
- [!] `append([1,2])` 会把整个列表塞进去，变成嵌套（`[[1,2]]`）
- [√] 删最后一个 `pop()`；删第一个 `pop(0)`；按值删 `remove(x)`
- [√] 复制一份用 `lst[:]` 或 `lst.copy()`
- [!] `sort()` / `reverse()` 是**原地修改**且**返回** `None` → `lst = lst.sort()` 是错的！
- [√] 想要**新列表**用内置函数：`sorted(lst)`（排序）、`list(reversed(lst))`（倒序）
- [!] 边遍历边删除会错位（见 3.12），要么倒着删、要么遍历副本
- [!] `lst = lst2` 不复制，只多起了一个名字

8.3 元组 tuple

定义：**有顺序、不能修改**的序列。

- [√] 适合装"**不该被改**的常量数据" —— 本作业的 `universities` 就是
- [!] 只含一个元素的元组要加逗号：`(1,)` 是元组，`(1)` 只是数字！
- [√] 元组最适合**解包**：`for name, city, line in universities:`
- [!] "不能改"只到第一层：元组里如果放了列表，那个列表**还是能改**的

```
t = (1, [2, 3])
t[1].append(4) # [√] 可以！因为改的是里面的 list，不是元组本身
```

8.4 集合 set

定义：一堆**不重复、无序**的元素。

- [√] 去重：`list(set(lst))`（**顺序会变**，题目允许才算合格）
- [√] 查重：`len(set(lst)) == len(lst)`
- [√] 判断在不在：`x in st`（比在 list 里找快得多）
- [!] **空集合只能写** `set()` —— `{}` 是空**字典**！
- [!] 不能放 `list` / `dict` / `set`（不可哈希）
- [!] 不能用下标：`st[0]` 会报 `TypeError`
- [√] 交集 `&`、并集 `|`、差集 `-`

8.5 字典 dict

定义：**键值对**的集合，靠 key 找 value。

- [√] 取可能不存在的键，用 `d.get(k, 默认值)`；直接写 `d[k]` 取不到会 `KeyError`
- [!] `k in d` 判断的是**键**，不是值
- [√] **计数三连**（背下来）：

```
if k in count: count[k] += 1
else: count[k] = 1
# 简写：count[k] = count.get(k, 0) + 1
```

- [√] 遍历方式：`for k in d` 得到**键**；`for k, v in d.items()` 得到**键和值**
- [√] Python 3.7+ 字典**会保留插入顺序**，但别把它当"有序容器"来用
- [!] 键必须**可哈希**（不能是 list）
- [√] 合并用 `d.update(other)`

8.6 range 与循环

- [√] `range(n)` 是 0 … n-1（**左闭右开**）
- [√] 倒序黄金公式：`for i in range(len(x) - 1, -1, -1)`
- [√] 只想遍历元素：`for x in lst`（**不需要下标就别用 range**）
- [√] 既要下标又要元素：`for i, x in enumerate(lst)`
- [√] 同时遍历两个列表：`for a, b in zip(list1, list2)`
- [!] 循环变量在循环结束后**仍然存在**（坑 3 的 `x` 就是这么来的）
- [!] `while` 里的条件必须**有机会变假**，否则死循环
- [!] 循环里别一边遍历一边删元素

8.7 函数与返回值

- [√] 函数的输入是**参数**，输出是 `return`；只认参数、不认全局
- [√] **多返回值**其实就是返回元组，调用时解包：

```
challenge, safe, not_rec = analyze_student(...)
```

- [√] 函数可以**当参数传**（回调）：`progress_function(0.05, final_analysis)`
- [√] 函数名**加括号才是调用**：`final_analysis` 是动作，`final_analysis()` 是执行
- [!] **没有** `return` **就返回** `None`（坑 3）
- [!] 函数里给变量赋值**不会**改到全局，除非写 `global`（不推荐）
- [!] 默认参数**别用可变对象**（`lst=[]` 会被所有调用共享，见 3.16）
- [√] 函数开头写 docstring：做什么 / 参数 / 返回什么

8.8 输出与格式

- [√] 拼多行报告用 `"\n".join(行列表)`，比反复 `+=` 干净
- [√] f-string 常用格式：`f"{x:3d}"`（右对齐 3 位）、`f"{p:.1f}"`（一位小数）
- [√] 进度条"原地刷新"三件套：`\r`（回车）+ `end=""`（不换行）+ `flush=True`（立刻输出）
- [!] `print()` 不传参数只输出一个**空行**
- [!] 中文/符号在不同终端宽度可能对不齐，别指望用空格硬对

8.9 调试与排错

- [√] 读报错**从下往上看**：最后一行是类型，倒数第二行是文件和行号
- [√] 不确定类型就 `print(type(x))`
- [√] 想看清空格/换行用 `print(repr(x))`（`strip()` 有没有生效一测就知道）
- [√] 二分定位：把确定没问题的部分先注释掉，缩小范围
- [√] 手推表格：把每一步的变量值写下来（坑 2 就是这么找到的）
- [!] **"不报错但结果错"最危险**（坑 1、坑 6），只能靠对照题目的"目标效果"发现
- [!] 变量名拼错导致 `NameError` 时，先看**是不是拼写**问题

8.10 写代码前的一句话自问

> 这个数据 **要顺序吗？要唯一吗？要按名字查吗？要能改吗？**
> 想清楚这四个问题，容器就选对了，这次作业 80% 的坑也就避开了。

附录：12 个任务 × 涉及知识点对照表

任务	要求	涉及知识点
任务 1	姓名去空格	`str.strip()`、字符串不可变、dict 取值与赋值
任务 2	兴趣标签去重	`set` 去重、无序性、hashable、`list()` 转换
任务 3	while 实现 clear	`while`、`len()`、`pop()`、**下标位移**（坑 2）
任务 4	for 倒序复制复杂列表	`for` + `range` 负步长、`isinstance`、切片 `[::-1]`（坑 9、10）
任务 5	大学冲 / 稳 / 保分档	tuple 解包、`for`、`if/elif/else` 顺序、`append`
任务 6	按兴趣推荐专业	dict 查值、`in` 判键、`set` 去重、`extend`（坑 1、4）
任务 7	函数返回多个结果	函数、参数、`return` 多返回值（元组）、作用域（坑 8）
任务 8	生成报告函数	函数返回字符串、`join`、列表收集行、f-string
任务 9	进度条 + 最终分析	函数作为参数（回调）、`return`、`print` 与 `return` 的区别（坑 3）
挑战 1	志愿表查重	`set`、`len()`、判断放在循环外（坑 5、6）
挑战 2	倒序输出志愿表	`range(len-1, -1, -1)`、下标访问、`print` 位置（坑 7、10）
挑战 3	统计专业出现次数	dict 计数模式、嵌套 `for`、遍历字典、`print` 位置（坑 7）

（本文件由 AI 助手根据我的作业错题整理，供复习使用。代码部分请以我自己重写的版本为准。）